

RAI-853: Advanced Robotics with ROS

Lec 05: Creating Your First ROS Package

Instructor:

Dr. Khawaja Fahad Iqbal

Deputy HoD (Department of Robotics & AI, SMME, NUST)

Scientific Director (Intelligent Robotics Lab, NCAI)



- **Improving the URDF with Xacro**
- **Xacro Macros**
- **Starting All nodes From the Terminal**
- **Creating a Package for Your URDF**
- **Writing a Launch File**
- **Python Launch File**

- **Improving the URDF with Xacro**
- Xacro Macros
- Starting All nodes From the Terminal
- Creating a Package for Your URDF
- Writing a Launch File
- Python Launch File

- The more complex your robot, the bigger the URDF. As you add more links and joints, you will end up having problems scaling your robot model.
- Also, what we have written so far is not so dynamic. All the values are hardcoded.
- Xacro is an additional ROS feature you can use to solve all those issues. With Xacro, your URDF files will become more dynamic and scalable.
- All serious ROS 2 projects use Xacro, so it's important to learn how to work with it.

- Before doing anything, let's make sure that Xacro is installed (it should already be there with all the previous packages we installed):

```
sudo apt install ros-<distro>-xacro
```

- Now, to use Xacro in your URDF file, you need to make two changes.
- First, change the file extension. The file is currently named my_robot.urdf. For Xacro, you will use the .xacro extension.
- A common practice is to use the .urdf.xacro extension for the main URDF file of your robot, so the file would be named my_robot.urdf.xacro (the important thing is to have .xacro at the end).

- Once you've changed the extension, open the file and modify the <robot> tag:

```
<robot name="my_robot"  
  xmlns:xacro="http://www.ros.org/wiki/xacro">
```

- Every time you want to use Xacro in a URDF file, you will have to add this xmlns:xacro argument.
- That's it for the setup. Now, to visualize the URDF in RViz, you will run the same command as before, with the new file name:

```
ros2 launch urdf_tutorial display.launch.py  
model:=/home/<user>/my_robot.urdf.xacro
```

- In programming, one of the first and most important things you learn is how to use variables. Variables do not exist in URDF, but you can use them with Xacro. Here, a variable is called a property.
- With Xacro properties, we will be able to specify values such as dimensions at the beginning of the file and use those properties inside the `<link>` and `<joint>` tags.
- This way, values and computations are not hardcoded. It will make things less confusing and more readable. Plus, if we ever need to modify a dimension of the robot, we just need to modify one value at the beginning of the file.

- Also, it's important to note that Xacro properties are considered constant variables.
- After you set their value, you won't modify them anymore.
- To declare and define a Xacro property, you will use the `<xacro:property>` tag and provide two arguments: name and value.

- Let's declare a few properties at the beginning of the file (inside the <robot> tag):

```
<xacro:property name="base_length" value="0.6" />
```

```
<xacro:property name="base_width" value="0.4" />
```

```
<xacro:property name="base_height" value="0.2" />
```

```
<xacro:property name="wheel_radius" value="0.1" />
```

```
<xacro:property name="wheel_length" value="0.05" />
```

- Those are all the values we need for computing everything else in the URDF.

- Then, to use a Xacro property, you simply have to write `${property_name}`.
- You can also do computations. For example, to multiply a value by 2.5, you will write `${property_name * 2.5}`.
- With this information, let's modify the content inside the **base_link** to remove any hardcoded value:

```
<geometry>  
  <box size="${base_length} ${base_width}  
${base_height}" />  
</geometry>  
  <origin xyz="0 0 ${base_height / 2.0}" rpy="0 0 0" />
```

- As you can see, we specify the box size with only properties. The most interesting part is how we compute the visual offset on the Z-axis.
- Writing $\{\text{base_height} / 2.0\}$ is much more explicit than just writing 0.1. Not only is it more dynamic, but we also have a better idea of what this computation is about.
- Imagine coming back to this URDF in six months and trying to figure out why the offset value is 0.1, without any context. With this property, there is no possible doubt.

- Let's now modify the visual for the **links of the right wheel and left wheel**:

```
<geometry>
  <cylinder
    radius="${wheel_radius}"
length="${wheel_length}"
  />
</geometry>
<origin xyz="0 0 0" rpy="${pi / 2.0} 0 0" />
```

- Finally, here is the code for the **caster wheel**:

```
<geometry>
```

```
<sphere radius="{wheel_radius / 2.0}" />
```

```
</geometry>
```

```
<origin xyz="0 0 0" rpy="0 0 0" />
```

- As you can see, we didn't define a `caster_wheel_radius` property. This is because the caster wheel radius needs to be half of the left and right wheel radius so that both wheels and caster wheels can touch the ground while making the robot stable.
- By using a Xacro property here, if we were to modify the wheel radius, then the caster wheel would automatically resize to make the robot stable.

- We have now modified all the links; let's change the values in the joint origins. For the **base_joint**, we have the following:

```
<origin xyz="0 0 ${wheel_radius}" rpy="0 0 0" />
```

- The **base_right_wheel_joint** is a bit more complex. However, once again, by writing this, we will make the computation more readable and less prone to errors in the future:

```
<origin xyz="${-base_length / 4.0} ${-(base_width  
+ wheel_length) / 2.0} 0" rpy="0 0 0" />
```

- The **base_left_wheel_joint** will be the same, except that the sign on the Y-axis is positive.
- We finish with the **base_caster_wheel_joint**:

```
<origin xyz="{base_length / 3.0} 0 ${-  
wheel_radius / 2.0}" rpy="0 0 0" />
```
- All those changes in the code didn't modify the robot model. When visualizing it in Rviz, it should look the same.

- To test that everything worked, **try modifying the values for some Xacro properties** at the beginning of the file.
- The robot model in RViz will have different dimensions, but it should still make sense.
- That's it for Xacro properties. This concept is not that hard to understand and apply, especially if you're already familiar with variables and constants..

- Improving the URDF with Xacro
- **Xacro Macros**
- Starting All nodes From the Terminal
- Creating a Package for Your URDF
- Writing a Launch File
- Python Launch File

- A Xacro macro is the equivalent of a function in programming.
- With Xacro, a macro works like a template: it's a piece of XML code that you can reuse with different values (parameters). A macro doesn't return anything.
- Macros are quite useful when you have to duplicate a link or a joint several times.
- Imagine a robot with four cameras. You can create a macro for the camera link, and then call the macro instead of re-writing the same code four times.

- With our robot model, we have almost the same code for the `right_wheel_link` and the `left_wheel_link`. The only difference is the name of the link. Let's create a macro for those links.
- To create a macro, you will use the `<xacro:macro>` tag and give a name as well as a list of params.
- You can specify zero, or as many parameters as you want—just separate them with a space.

- Here is an example:

```
<xacro:macro name="wheel_link" params="prefix">
  <link name="${prefix}_wheel_link">
    <visual>
      <geometry>
        <cylinder radius="${wheel_radius}"
          length="${wheel_length}" />
      </geometry>
      <origin xyz="0 0 0" rpy="${pi / 2.0} 0 0" />
      <material name="gray" />
    </visual>
  </link>
</xacro:macro>
```

- This piece of code won't do anything by itself. We need to call it, just like you would call a function.
- To call a macro, you will write the following:

```
<xacro:name param1="value" param2="value" />
```

- Remove the right_wheel_link and the left_wheel_link, and write this instead:

```
<xacro:wheel_link prefix="right" />
```

```
<xacro:wheel_link prefix="left" />
```

- The "right" value in the prefix parameter will be applied to the link name inside the macro, making the name `right_wheel_link`. The same thing applies for the left wheel.
- As you can see, macros can help you **reduce code duplication**. In this example, the benefit is not as big, but if you need to duplicate some links or joints more than three times, then macros can be extremely useful.
- Also, if you are creating part of a URDF that will be used by other people, writing a macro can help them to easily integrate your code into theirs, and customize it with the different parameters they can give as input.

- Thanks to Xacro, you can split your URDF into several files.
- This is useful to **separate different parts** of your robot, for example, the main core base, and extra sensors that you add on top.
- If you **combine two robots**, for example, a robotic arm on top of a mobile robot, you can create one URDF for each robot and combine them into a third one.
- Another benefit is **collaboration**. Creating a macro inside one file that other developers can include will make it easier for them to work with your code.

- Coming back to our URDF, let's split the file into three:
1. **common_properties.xacro:** This will contain the material tags and other properties that could apply to any part of our robotics application
 2. **mobile_base.xacro:** This file will contain the properties, macros, links, and joints that are specific to the mobile base
 3. **my_robot.urdf.xacro:** In this main file, we include the two previous files.

- By doing this, we will make the URDF more dynamic and easier to modify in the future.
- If you want to combine several mobile bases or add other robots or sensors, you can create more Xacro files that you include into the main one (my_robot.urdf.xacro).
- Now, create the two additional files, and let's see how to organize the code. Make sure to put all three files in the same directory.

- Let's start with common_properties.xacro. Here is the first thing to write inside this file:

```
<?xml version="1.0"?>
```

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
```

```
</robot>
```

- All Xacro files must have this code, with a <robot> tag containing the xmlns:xacro attribute.

- Then, inside the `<robot>` tag, you can copy and paste the two `<material>` tags we have previously written. You can then remove those tags from `my_robot.urdf.xacro`.
- In `mobile_base.xacro`, you will also start the file with the `<?xml>` and `<robot>` tags, like we did in `common_properties.xacro`.
- Then, you can copy and paste all the `<xacro:property>`, `<xacro:macro>`, `<link>`, and `<joint>` tags that are related to the mobile base, which are basically all the tags left.

- In my_robot.urdf.xacro, we have nothing left, only the `<?xml>` tag and the `<robot>` tag that contains the name and `xmlns:xacro` attributes.
- To include a Xacro file inside another file, you will write a `<xacro:include>` tag and provide the path to the file with the filename attribute.
- Here is the final content for my_robot.urdf.xacro:

```
<?xml version="1.0"?>  
<robot name="my_robot"  
xmlns:xacro="http://www.ros.org/wiki/xacro">  
<xacro:include filename="common_properties.xacro" />  
<xacro:include filename="mobile_base.xacro" />  
</robot>
```

- Our URDF is now split into several files and uses Xacro properties and macros.
- With those modifications, we didn't change anything about the robot model, but we made the URDF more dynamic and scalable, as well as easier to read.

- Improving the URDF with Xacro
- Xacro Macros
- **Starting All nodes From the Terminal**
- Creating a Package for Your URDF
- Writing a Launch File
- Python Launch File

- Before we package the application and write a launch file, let's start all the nodes we need in the terminal.
- Doing this is a best practice so that you can make sure your application is working properly.
- Then, creating the package and the launch file will be easier, as you already know all the elements you need to include.

- Let's publish the TFs. For that, we will open two terminals.
- In Terminal 1, run the following command:

```
$ ros2 run robot_state_publisher robot_state_publisher --  
ros-args -p robot_description:="$(xacro  
/home/<user>/my_robot.urdf.xacro)"
```

```
[INFO] [1772121661.941109320]  
[robot_state_publisher]: Robot initialized
```

```
--
```

```
[robot_state_publisher]: got segment base_footprint
```

```
[robot_state_publisher]: got segment base_link
```

```
[robot_state_publisher]: got segment caster_wheel_link
```

```
[robot_state_publisher]: got segment left_wheel_link
```

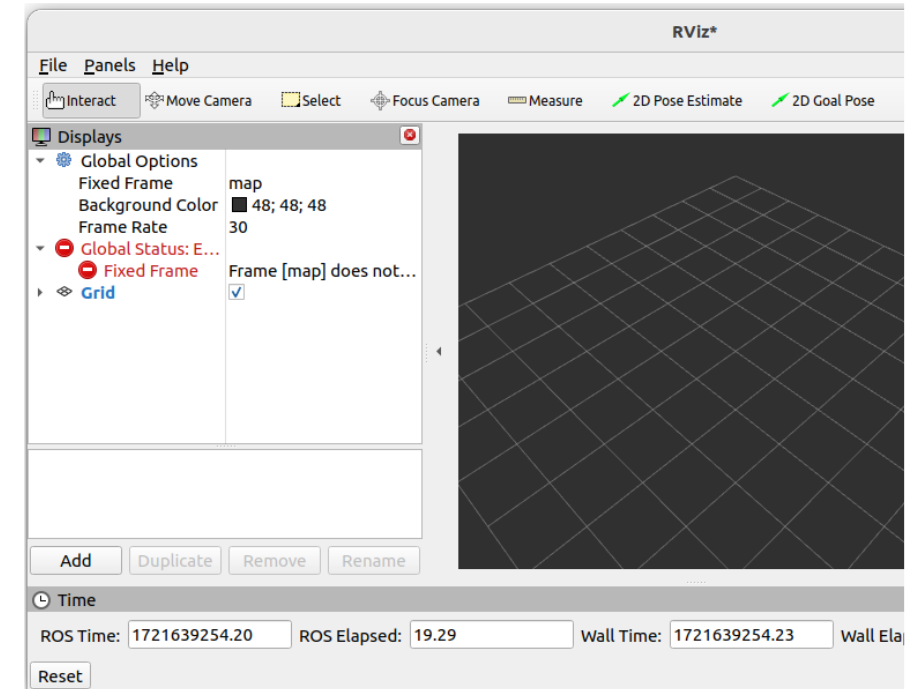
```
[robot_state_publisher]: got segment right_wheel_link
```


- If the previous command has executed successfully, that means that `robot_state_publisher` has been started with the URDF, and it's ready to publish on the `/tf` topic.
- Now, we need to add a publisher on the `/joint_states` topic. We will use the `joint_state_publisher_gui` executable from the same package (note the extra `_gui` suffix, which means graphical user interface).
- In Terminal 2, run the following command:

```
$ ros2 run joint_state_publisher_gui  
joint_state_publisher_gui
```

- This will open the Joint State Publisher window that we previously used when experimenting with TFs and URDF.
- The values you see on the cursors will be published on the `/joint_states` topic and will be received by `robot_state_publisher`.
- That's basically all we need. This will become the backbone of your ROS 2 application—of course, we will need to package this nicely and start it from a launch file.

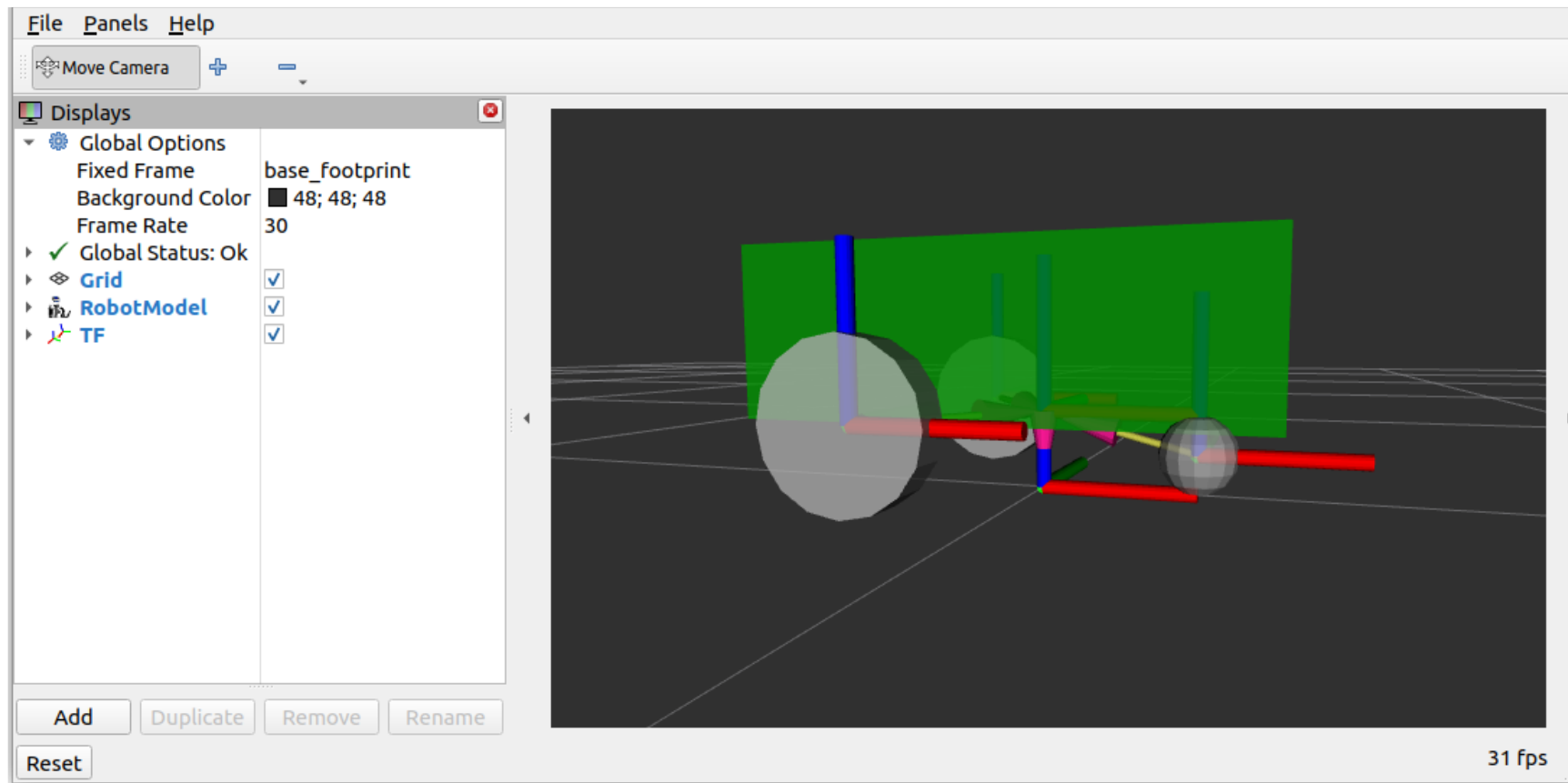
- Let's start RViz and see how to visualize the robot model as well as the TFs.
- Keep the `robot_state_publisher` and `joint_state_publisher` nodes running.
- Then, in Terminal 3, run the following command:
`$ ros2 run rviz2 rviz2`
- This will open RViz, but, as you can see, there is no robot model, and we have some errors on the left menu (Frame [map] does not exist).



- We need to do a bit of configuration to correctly visualize the robot model and the TFs. Then, we will be able to save this configuration and reuse it the next time we start RViz.
- Follow these steps to configure RViz:
 1. In the left menu, **Global Options | Fixed Frame**, change **from map to base_footprint**. After that, Global Status: Error should change to Global Status: OK.
 2. Click on the **Add button** on the left, scroll down, and double-click on **RobotModel**. You will have a new menu on the left side of RViz.

3. Open this new **RobotModel** menu, find **Description Topic**, and click on the **empty space on the right side of the menu** (this one is a bit tricky to find). You should see a drop-down menu; here, select **/robot_description**. After this, the robot model should appear on the screen.
4. Click on the Add button again, scroll down, and double-click on **TF**. This will open a new menu, and you will see the TFs appear on the screen.
5. If you want to see through the model, like we did before, open RobotModel, and **reduce the Alpha** (transparency) value from 1 to 0.8, for example.
6. You can **remove the extra menus** on the right (Views) and at the bottom (Time) to get more space for the robot.

- With all those settings, you should see the robot model and TFs the same way we did when we previously visualized the URDF with the urdf_tutorial package.



- You will need to repeat those steps every time you start RViz. To avoid doing this, we will save the configuration.
- Click on File | Save Config As. Let's name the file **urdf_config.rviz** (for these files, use the .rviz extension), and place it inside your **home directory** for now.
- Click on File | Save Config to ensure that your file has been saved in the correct configuration.
- Make sure you can see the file, using a file manager or the terminal. If you didn't save the file correctly, you will need to manually do the full configuration again.

- Once the file is saved, you can stop Rviz (Ctrl + C in the terminal).

- Then, when you start RViz again, you can add an extra -d argument with the path to the configuration file:

```
$ ros2 run rviz2 rviz2 -d /home/<user>/urdf_config.rviz
```

- This will start RViz exactly like you saved it: same menus, same view, same zoom, and so on.

- Improving the URDF with Xacro
- Xacro Macros
- Starting All nodes From the Terminal
- **Creating a Package for Your URDF**
- Writing a Launch File
- Python Launch File

- All the files we have created are now in our home directory. It's time to create a ROS 2 package and move all the files into the right place so that they can be installed and used in our ROS 2 workspace.
- We will start by creating a package dedicated to the robot model. Then, we will install all the files required for this application.
- This will allow us to use the URDF and RViz files when we write a launch file, to start all the nodes we have seen previously.

- To name the package, we use the robot's name, followed by `_description`. This is a common convention used by lots of ROS developers. By using this naming format, you make it clear that this package will contain the URDF files for your robot. This will make collaboration easier.
- To create this new package, we use the `ament_cmake` build type, just as if we were creating a C++ package, and we don't specify any dependencies. Let's create the package:

```
$ cd ~/ros2_ws/src/
```

```
$ ros2 pkg create my_robot_description --build-type  
ament_cmake
```

- For now, this package is a standard C++ ROS 2 package. However, we won't write any nodes inside.
- We just need the package to install our robot model. Thus, you can remove the src and include directories:

```
$ cd my_robot_description/  
$ rm -r include/ src/
```
- At this point, your package will only contain two files: package.xml and CMakeLists.txt.

- Open the workspace with an IDE. If you're using VS code, run the following:

```
$ cd ~/ros2_ws/src/
```

```
$ code .
```

- To clean things a bit more, we will simplify the CMakeLists.txt file. Remove the comments after the find_package(ament_cmake REQUIRED) line, and, as we don't need that now, remove the if(BUILD_TESTING) block.
- Make sure you keep the ament_package() instruction, which should be the last line of the file.

- Now that we have our robot_description package, let's install all the files we need.
- To install a file, we will follow this process:
 1. Create a folder to host the files.
 2. Add an instruction in CMakeLists.txt to install the folder.
- Here, we will see how to install the URDF files, custom meshes, and the RViz configuration.

- To install our Xacro and URDF files, go inside the my_robot_description package and create a new folder named urdf:

```
$ cd ~/ros2_ws/src/my_robot_description/
```

```
$ mkdir urdf
```

- You can now move all three Xacro files inside this urdf folder: common_properties.xacro, mobile_base.xacro, and my_robot.urdf.xacro.

- Everything should work fine, but to make the include paths more robust, let's modify the `<xacro:include>` tags inside `my_robot.urdf.xacro`:

```
<xacro:include filename="$(find  
my_robot_description)/urdf/common_properties.xac  
ro" />
```

```
<xacro:include filename="$(find  
my_robot_description)/urdf/mobile_base.xacro" />
```

- Instead of providing only the relative path (which should still work in this case), **we provide the absolute path** to where the files will be installed, using the `find` keyword before the package name.

- Now, **open CMakeLists.txt**, and add those instructions to install the urdf folder:

```
find_package(ament_cmake REQUIRED)
install(
  DIRECTORY urdf
  DESTINATION share/${PROJECT_NAME}/
)
ament_package()
```

- This will install the urdf folder inside a share directory when you build the package. It will allow any package from the workspace to find the URDF for your robot.

- By installing the RViz configuration, when we start RViz later on from a launch file, we can use this configuration directly from the package.
- Create an rviz folder inside the package:

```
$ cd ~/ros2_ws/src/my_robot_description/  
$ mkdir rviz
```
- Move the urdf_config.rviz file into this new rviz folder.

- Now, to install the folder, add its name to the install() instruction, inside CMakeLists.txt:

```
install(  
  DIRECTORY urdf rviz  
  DESTINATION share/${PROJECT_NAME}/  
)
```

- We now have all the files we need for this package. Before we install them (with colcon build), let's add a launch file so we can start all the required nodes to publish the TFs as well as visualize the robot model in RViz.

- Improving the URDF with Xacro
- Xacro Macros
- Starting All nodes From the Terminal
- Creating a Package for Your URDF
- **Writing a Launch File**
- Python Launch File

- The my_robot_description package is finished, but we will add a launch file so we can start all the nodes and parameters simultaneously.
- This way, we can publish TFs and visualize the robot in RViz. This will also be a good practice exercise on launch files.
- We will also reuse part of the code later when we build the Gazebo simulation for the robot.

- We will write the launch file first with XML. This will be an example of how XML launch files can be easier to write than Python ones.
- Before writing any launch file, we first need to create a launch folder, where we will put all our launch files for the my_robot_description package.
- To create and install a launch folder, first we will create the folder:

```
$ cd ~/my_robot_ws/src/my_robot_description/
```

```
$ mkdir launch
```

- Then, to specify the instructions to install this folder, it's as easy as adding the folder name inside CMakeLists.txt:

```
install(  
  DIRECTORY urdf rviz launch  
  DESTINATION share/${PROJECT_NAME}/  
)
```

- As the install() instruction was already configured, if we want to install a new folder, we just add its name after the other ones.

- Once this is done, create a new launch file inside the folder. We will name it display.launch.xml:

```
$ cd launch/
```

```
$ touch display.launch.xml
```

- Now, let's write the content for this launch file.

- In this launch file, we will simply start the following nodes and parameters:
 - robot_state_publisher with the URDF as the robot_description parameter. This node will publish on the /tf topic.
 - joint_state_publisher to publish on the /joint_states topic.
 - rviz2, as we also want to visualize the robot model.
- We already know how to start those nodes from the terminal; now, all we have to do is add them inside one launch file.

- In the launch file, first open and close a `<launch>` tag:

`<launch>`

`</launch>`

- Now, make sure to write all the following lines inside this `<launch>` tag.
- You can also add an indentation (four spaces) for those lines.

- As we will need to find the path of the URDF file and the RViz configuration file, we add two variables at the beginning of the launch file, to make the file cleaner and more readable.
- Also, if you have to modify those values later, you know that you just have to modify the variables at the top of the file.

- You will use the <let> tag with two arguments: name and value. Let's add the two variables we need:

```
<let name="urdf_path" value="$(find-pkg-share  
my_robot_description)/urdf/my_robot.urdf.xacro" />
```

```
<let name="rviz_config_path" value="$(find-pkg-share  
my_robot_description)/rviz/urdf_config.rviz" />
```

- Then, to use a variable, you can write \$(var name).

- Now, let's start all the nodes we need, one by one. Here is the code for the robot_state_publisher node:

```
<node pkg="robot_state_publisher"  
exec="robot_state_publisher">  
  <param name="robot_description"  
    value="$(command 'xacro $(var urdf_path)')" />  
</node>
```

- Nothing special here; we use the same values as in the command we previously ran in the terminal (See slide 32).
- To specify a command to run in an XML launch file, you can use `$(command '...')`.

- Next, we can start the joint_state_publisher node.
- Here, we use the executable with the _gui suffix to get a graphical window with cursors to move the joints:

```
<node pkg="joint_state_publisher_gui"  
exec="joint_state_publisher_gui" />
```

- This node is easy to write, as we don't need to provide anything else except the package and executable.

- Let's finish with the RViz node:

```
<node pkg="rviz2" exec="rviz2" args="-d $(var  
rviz_config_path)" />
```

- In this node, we provide the saved RViz configuration file, using the -d option.

- The launch file is now complete. We can build the workspace to install all the files and folders that we've added:

```
$ cd ~/ros2_ws/
```

```
$ colcon build --packages-select my_robot_description
```

- Then, don't forget to source the workspace (**source install/setup.bash**), and you can start your new launch file:

```
$ ros2 launch my_robot_description display.launch.xml
```


- After running this command, you should see your robot model in RViz with the TFs.
- If you list nodes and topics on the terminal, you should be able to find everything that we have started.
- The my_robot_description package is now complete. We have installed all the files that we need: URDF, custom meshes, RViz configuration, and a launch file to publish TFs and visualize the robot model.

- Improving the URDF with Xacro
- Xacro Macros
- Starting All nodes From the Terminal
- Creating a Package for Your URDF
- Writing a Launch File
- **Python Launch File**

- Let's now create the Python launch file. To do that, create a new file inside the **launch folder** of the `my_robot_description` package.
- You can name this file **`display.launch.py`**—the same name as for the XML launch file, but with a Python extension.
- There is no need to add any extra configuration in the package, as `CMakeLists.txt` already contains the instructions to install the launch folder.

- Let's start to analyze this Python file with the import lines:

```
from launch import LaunchDescription
```

```
from launch_ros.parameter_descriptions import  
ParameterValue
```

```
from launch_ros.actions import Node
```

```
from launch.substitutions import Command
```

```
import os
```

```
from ament_index_python.packages import  
get_package_share_path
```

- All the code we have from now on will be inside this function (with an indentation).
- We create the two variables for the URDF and RViz path:

```
urdf_path = os.path.join(  
get_package_share_path('my_robot_description'),  
'urdf', 'my_robot.urdf.xacro')  
  
rviz_config_path = os.path.join  
(get_package_share_path('my_robot_description'),  
'rviz', 'urdf_config.rviz' )
```

- To make things a bit more readable, we also create a variable for the robot_description parameter:

```
robot_description =  
ParameterValue(Command(['xacro ', urdf_path]),  
value_type=str)
```

- Now, we start the three nodes:

```
robot_state_publisher_node = Node(  
    package="robot_state_publisher",  
    executable="robot_state_publisher",  
    parameters=[{'robot_description': robot_description}])
```

```
joint_state_publisher_gui_node = Node(  
    package="joint_state_publisher_gui",  
    executable="joint_state_publisher_gui")
```

```
rviz2_node = Node(package="rviz2", executable="rviz2",  
    arguments=['-d', rviz_config_path])
```

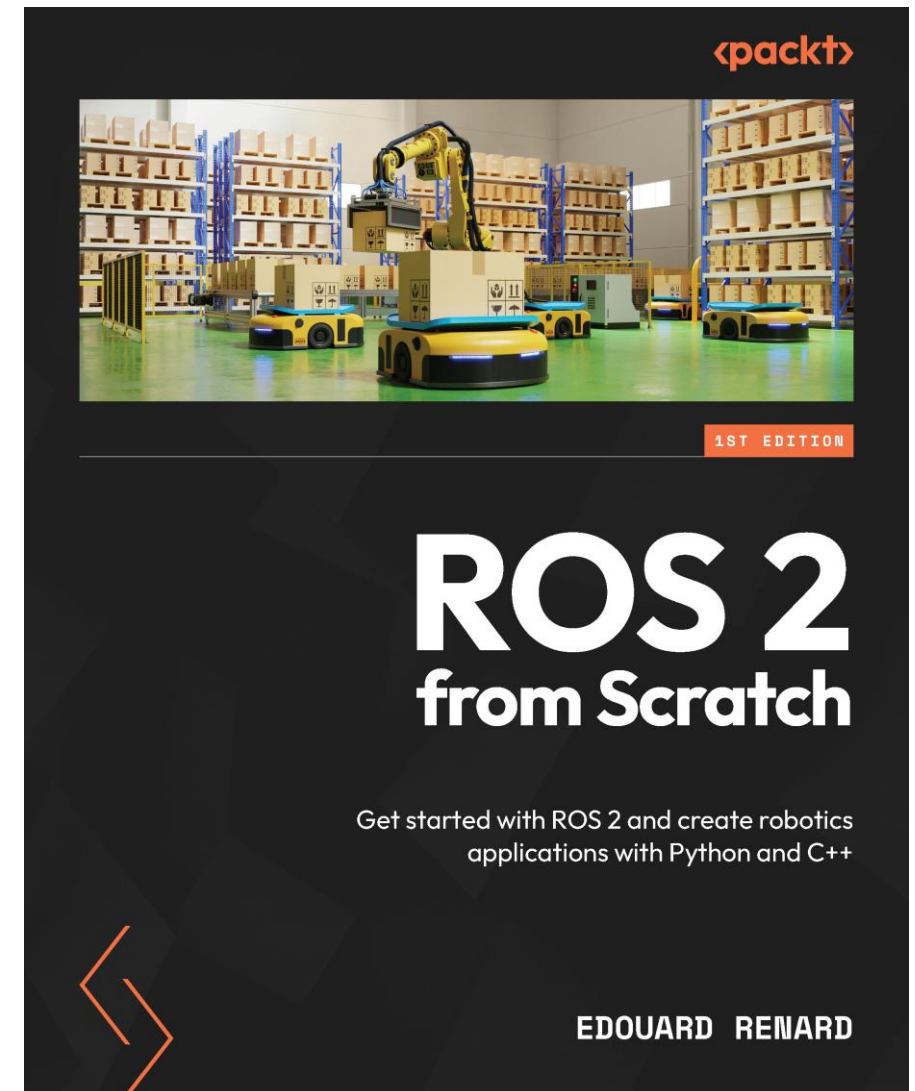
- Finally, we need to return a LaunchDescription object with all the nodes we want to start:

```
return LaunchDescription([  
    robot_state_publisher_node,  
    joint_state_publisher_gui_node,  
    rviz2_node  
])
```


- As compared to XML launch file, the code for Python launch file is much longer: 38 lines for Python versus 15 lines for XML. We could optimize the spaces, but even with that, there would still be a factor of two.
- Python syntax is also more complex and not really intuitive.
- You can run the Python launch file to see if everything is in order:

```
$ ros2 launch my_robot_description display.launch.py
```

- ROS 2 from Scratch by Edouard Renard
- Chapter 11 and 12



Thank you for your attention.

Any Questions?

